



universität
uulm

University of Ulm
Faculty of Mathematics and Economics

American Options: Random Trees

Seminar Paper

Practical Financial Engineering

Viktoriia Behisheva
Anastasiia Savelieva

03.07.2025

Contents

1	Introduction	1
2	Theory related to results	2
2.1	American Option Problem Setup	2
2.2	Dynamic Programming Recursion	3
2.3	Random Tree Method	3
2.3.1	Confidence Intervals	5
2.3.2	High Estimator	6
2.3.3	Low Estimator	7
2.3.4	Asymptotic Convergence	8
2.4	Depth-First Processing (DFS)	8
3	Implementation	10
4	Simulation results	11
4.1	Single Random-Tree Valuation of an American Call Option	11
4.2	Effect of Branching Factor b	12
4.2.1	Distribution of CI Widths	12
4.2.2	High/Low Trajectories	13
4.3	Effect of Number of Trees n	13
4.3.1	CI Widths vs. n	13
5	Conclusions	15
	Bibliography	16
	Appendix 1	17
	Appendix 2	20

1 Introduction

American options are a class of financial derivatives that grant the holder the right, but not the obligation, to buy (in the case of a *call option* $\max(S_t - K, 0)$) or sell (in the case of a *put option* $\max(K - S_t, 0)$) an underlying asset at a predetermined *strike price* K at any time up to or on the option's maturity date ($t \leq T$), where S_t denotes the price of the underlying asset at time t . This early-exercise feature makes American options more flexible and often more valuable than their European counterparts, which can only be exercised at maturity T . However, this same flexibility significantly complicates their valuation.

The core challenge in pricing American options lies in determining the optimal exercise strategy, which involves solving an optimal stopping problem. At each possible exercise time, the holder must choose between exercising the option immediately or continuing to hold it in anticipation of a better payoff. This decision depends on various factors, including the evolution of the underlying asset price and prevailing interest rates. Mathematically, the problem is often approached through backward dynamic programming, where the option value is computed recursively from the final time step back to the present. For simple settings, discrete-time models such as the binomial or trinomial tree are commonly used, but they quickly become computationally infeasible in high-dimensional or path-dependent cases due to the exponential growth in the number of possible scenarios.

To address these limitations, simulation-based methods, especially Monte Carlo techniques have gained popularity. Among them, Stochastic Mesh and Regression-Based approaches (such as the Least-Squares Monte Carlo method) have shown success in high-dimensional settings. However, these methods often require careful function approximation or the construction of specific transition probability weights, which can introduce bias or instability.

This presentation focuses on a powerful alternative: the Random Tree method proposed by Glasserman and Broadie, which was first time introduced in 1997 and then developed in 2004. This method blends the intuitive structure of decision trees with the flexibility of Monte Carlo simulation. It requires only the ability to simulate paths of the underlying asset, without relying on specific model assumptions or analytical pricing formulas. A key strength of the method is its provision of two es-

timators—an upper and a lower bound on the true option value—between which the actual price lies with high probability. This feature offers a built-in measure of accuracy, setting it apart from many simulation methods that yield only point estimates.

In the following sections, we will examine the theoretical foundation of the Random Tree method, its algorithmic implementation, and its performance in practice. Simulations conducted in R demonstrate its application to Bermudan-style American options on a single underlying asset. We conclude by analyzing how branching factor and sample size affect estimator accuracy, supported by numerical experiments and visualizations. The seminar also includes a practical implementation of the method using depth-first processing, highlighting both its computational feasibility and limitations.

2 Theory related to results

2.1 American Option Problem Setup

An American option allows the holder to exercise the contract at any time up to and including maturity. Pricing such options naturally leads to an optimal stopping problem.

Let $\mathcal{T}$ denote the set of all stopping times taking values in $\{0, 1, \dots, m\}$, where m is the number of discrete exercise opportunities. For $\tau \in \mathcal{T}$, let $U(\tau)$ denote the payoff received upon exercising at time τ . Then, the price of the American option is given by:

$$V = \sup_{\tau \in \mathcal{T}} \mathbb{E}[U(\tau)],$$

where the supremum is taken over all admissible stopping times in $\mathcal{T}$.

Let $X(t)$ denote the Markov process describing the evolution of the underlying assets. The payoff function is denoted by $h(X(t))$, where $h : \mathbb{R}^d \rightarrow \mathbb{R}$ is typically of the form $h(x) = (K - x)^+$ for an American put option with strike K .

In practice, American options are often approximated by Bermudan options, which restrict exercise to a finite set of dates $t_0, t_1, \dots, t_m$. This discretization transforms the continuous-time stopping problem into a tractable, discrete-time problem suitable for numerical methods.

2.2 Dynamic Programming Recursion

To solve the Bermudan option pricing problem, we use backward dynamic programming. Let $V_i(x)$ be the value of the option at time t_i given $X_i = x$. Then, the value function satisfies the following recursive relationship:

$$V_i(x) = \max \{h_i(x), \mathbb{E}[V_{i+1}(X_{i+1}) \mid X_i = x]\}, \quad i = m-1, m-2, \dots, 0,$$

with terminal condition: $V_m(x) = h_m(x)$.

This recursion expresses the fundamental trade-off at each decision point: either exercise now and receive the immediate payoff $h_i(x)$, or continue holding the option, anticipating a higher expected future value.

In this formulation, the value functions $V_i(x)$ and payoffs $h_i(x)$ are expressed in time-0 dollars. This implicitly embeds all discount factors within the expectation operator. Specifically, the term $\mathbb{E}[V_{i+1}(X_{i+1}) \mid X_i = x]$ incorporates the discount factor from t_i to t_{i+1} through the relationship $V_{i+1}(X_{i+1}) = D_{0,i+1}(X_{i+1})\tilde{V}_{i+1}(X_{i+1})$, where $\tilde{V}_{i+1}$ denotes the value in time- t_{i+1} dollars and $D_{0,i+1}$ is the discount factor from time 0 to t_{i+1} . Equivalently, if values are expressed in time- t_i dollars, the recursion explicitly includes a discount factor:

$$\tilde{V}_i(x) = \max \{\tilde{h}_i(x), \mathbb{E}[D_{i,i+1}(X_{i+1})\tilde{V}_{i+1}(X_{i+1}) \mid X_i = x]\}, \quad i = m-1, \dots, 0,$$

where $D_{i,i+1}(X_{i+1})$ discounts cashflows from t_{i+1} to t_i and $\tilde{h}_i(x)$ is the immediate exercise payoff at time t_i , expressed in time- t_i dollars (undiscounted from t_i). This discount factor may take the form

$$D_{i,i+1}(X_{i+1}) = \exp\left(-\int_{t_i}^{t_{i+1}} r(u)du\right),$$

reflecting the time value of money under a short-rate process $r(u)$. The formulations are equivalent via the mappings $V_i(x) = D_{0,i}(x)\tilde{V}_i(x)$ and $h_i(x) = D_{0,i}(x)\tilde{h}_i(x)$.

2.3 Random Tree Method

The random tree method is a simulation-based approach for approximating the value of American options by building a tree of possible future paths of the underlying asset. The idea is to simulate multiple future scenarios from each point in time,

forming a tree-like structure that represents the evolution of the asset price over discrete time steps.

At each time step i , starting from a current state x , we generate b independent future states $X_{i+1}^{(1)}, \dots, X_{i+1}^{(b)}$, where each successor is sampled according to the transition distribution of the stochastic process (often assumed to follow a Markov process). This process is repeated at each new node, resulting in a tree with branching factor b and depth m , where m is the total number of time steps. Since each node spawns b children, the total number of nodes grows exponentially with m .

The purpose of this tree is to estimate the conditional expectations needed for the backward dynamic programming recursion used in pricing American options. Instead of computing exact expectations (which is often intractable), we use sample averages over the simulated paths to approximate these values.

To describe the tree structure in more detail, let X_0 be the initial state of the underlying process. From X_0 , we simulate b independent states $X_1^1, X_1^2, \dots, X_1^b$, each distributed according to the law of X_1 . Then, from each X_1^j , we simulate b new successors $X_2^{j1}, \dots, X_2^{jb}$, based on the conditional distribution of X_2 given $X_1 = X_1^j$. This branching process continues recursively.

A generic node at time i is denoted by $X_i^{j_1 j_2 \dots j_i}$, where the superscripts indicate the sequence of branches taken to reach that node: the j_1 -th branch from X_0 , the j_2 -th branch from the next node, and so on.

Although the branching factor b can vary between time steps, it is often assumed to be fixed for simplicity and consistency in implementation.

Illustration. Figure 1 illustrates the construction of a random tree with branching factor $b = 3$ and depth $m = 2$, representing the possible evolutions of the underlying asset price over time in the context of option pricing. The model is based on the following parameters: initial stock price $S_0 = 100$, strike price $K = 100$, risk-free interest rate $r = 0.05$, volatility $\sigma = 0.2$, time to maturity $T = 1$ (in years).

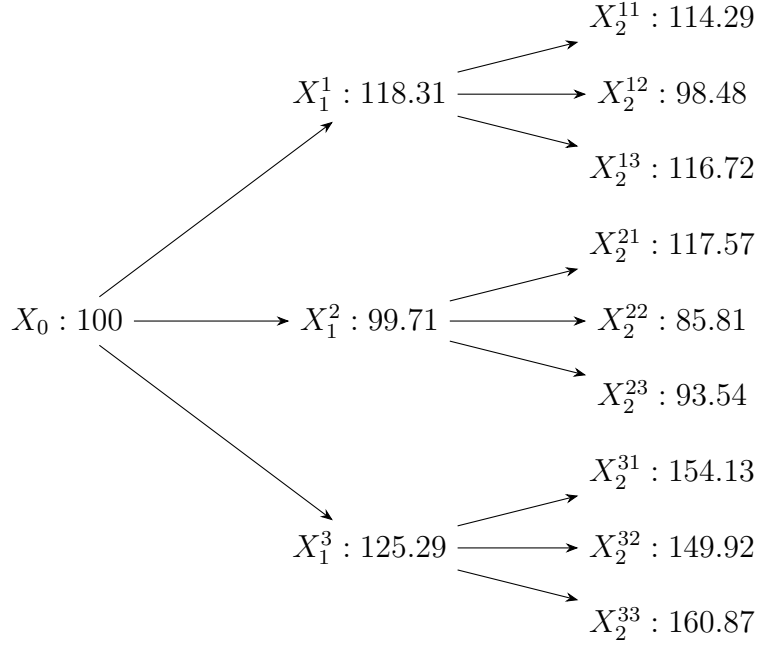


Figure 1: Random tree

2.3.1 Confidence Intervals

The random tree method is used to estimate the price of an American option by constructing two estimators: a high estimator $V_n(b)$ and a low estimator $v_n(b)$. These estimators allow us to form a confidence interval around the true option price. Specifically, we define half-widths $H_n(b)$ and $L_n(b)$ corresponding to the standard errors of $V_n(b)$ and $v_n(b)$, respectively, and construct a conservative interval. Let $\{V_n^{(j)}(b)\}_{j=1}^N$ and $\{v_n^{(j)}(b)\}_{j=1}^N$ be N independent replications of the high and low estimators, respectively. Define their sample means and standard deviations by

$$\begin{aligned} \bar{V}_n(b) &= \frac{1}{N} \sum_{j=1}^N V_n^{(j)}(b), & s_V^2(n,b) &= \frac{1}{N-1} \sum_{j=1}^N (V_n^{(j)}(b) - \bar{V}_n(b))^2, \\ \bar{v}_n(b) &= \frac{1}{N} \sum_{j=1}^N v_n^{(j)}(b), & s_v^2(n,b) &= \frac{1}{N-1} \sum_{j=1}^N (v_n^{(j)}(b) - \bar{v}_n(b))^2. \end{aligned}$$

By the Central Limit Theorem, an approximate two-sided $(1 - \alpha)$ -level confidence interval for $\mathbb{E}[V_n(b)]$ is

$$\bar{V}_n(b) \pm z_{1-\alpha/2} \frac{s_V(n,b)}{\sqrt{N}}. \implies H_n(b) = z_{1-\alpha/2} \frac{s_V(n,b)}{\sqrt{N}}.$$

Similarly, a $(1 - \alpha)$ -level CI for $\mathbb{E}[v_n(b)]$ is

$$\bar{v}_n(b) \pm z_{1-\alpha/2} \frac{s_v(n,b)}{\sqrt{N}} \implies L_n(b) = z_{1-\alpha/2} \frac{s_v(n,b)}{\sqrt{N}}.$$

Finally, since $v_n(b) \leq V \leq V_n(b)$ in expectation, a conservative $(1 - \alpha)$ confidence band for the true option value V is

$$[\bar{v}_n(b) - L_n(b), \bar{V}_n(b) + H_n(b)]$$

Here, we take the lower bound from the low estimator and the upper bound from the high estimator. Under mild assumptions, this interval contains the true option price with at least 90% probability, and under symmetry assumptions it can reach 95% confidence. By increasing N (and/or the tree-branching b) both half-widths $L_n(b), H_n(b) \rightarrow 0$ and the interval collapses onto the true price.

2.3.2 High Estimator

The high estimator is computed by applying the dynamic programming recursion over the full random tree using sample averages to estimate expectations.

Write $\widehat{V}_i^{j_1 \dots j_i}$ for the value of the high estimator at node $X_i^{j_1 \dots j_i}$. At the terminal nodes we set

$$\widehat{V}_m^{j_1 \dots j_m} = h_m(X_m^{j_1 \dots j_m}).$$

At each preceding node, compute the estimator as the maximum of the immediate exercise value and the estimated continuation value. Working backward, we then set:

$$\widehat{V}_i^{j_1 \dots j_i} = \max \left\{ h_i(X_i^{j_1 \dots j_i}), \frac{1}{b} \sum_{j=1}^b \widehat{V}_{i+1}^{j_1 \dots j_i j} \right\}.$$

Now observe that the function $f(y) = \max\{h_i(X_i), y\}$ is convex in y . Hence by Jensen's inequality,

$$\mathbb{E}[\widehat{V}_i | X_i] = \mathbb{E}[f(Y) | X_i] \geq f(\mathbb{E}[Y | X_i]) = \max \left\{ h_i(X_i), \mathbb{E}[\widehat{V}_{i+1} | X_i] \right\} = V_i(X_i).$$

Since this holds at every node, taking the expectation at the root gives $\mathbb{E}[\widehat{V}_0] \geq V_0(X_0)$, meaning this estimator is biased upward. Intuitively, using the same sample to both estimate continuation values and decide whether to exercise “peeks into the future”, producing an upward bias.

2.3.3 Low Estimator

To obtain a lower-biased estimator, we partition the b child nodes into two groups. One group is used to make the early exercise decision; the other is used to estimate the continuation value. For each node, the decision to exercise or continue is based on the estimated continuation value from one group, and the payoff is assigned based on the other.

The estimator is defined as follows: At all terminal nodes, the estimator is equal to the payoff at that node:

$$\hat{v}_m^{j_1 j_2 \dots j_m} = h_m(X_m^{j_1 j_2 \dots j_m}).$$

At node $j_1 j_2 \dots j_i$ at time step i and for each $k = 1, \dots, b$ set:

$$\hat{v}_{ik}^{j_1 j_2 \dots j_i} = \begin{cases} h_i(X_i^{j_1 j_2 \dots j_i}) & \text{if } \frac{1}{b-1} \sum_{\substack{j=1 \\ j \neq k}}^b \hat{v}_{i+1}^{j_1 j_2 \dots j_i j} \leq h_i(X_i^{j_1 j_2 \dots j_i}); \\ \hat{v}_{i+1}^{j_1 j_2 \dots j_i k} & \text{otherwise;} \end{cases}$$

then set

$$\hat{v}_i^{j_1 j_2 \dots j_i} = \frac{1}{b} \sum_{k=1}^b \hat{v}_{ik}^{j_1 j_2 \dots j_i}.$$

The estimator of the option price at the current time and state is $\hat{v}_0$.

Key Intuition: This estimator mimics an adaptive exercise strategy by testing exercise decisions under uncertainty (using partitioned subsets of children). The averaging ensures it represents a feasible strategy, providing a mathematical lower bound for the true option price.

Why is it biased low? Consider the simplified setting where we want to estimate

$$\max(a, \mathbb{E}[Y]), \text{ but we use: } \hat{v} = \begin{cases} a & \text{if } \bar{Y}_1 \leq a, \\ \bar{Y}_2 & \text{otherwise,} \end{cases}$$

where $\bar{Y}_1$ and $\bar{Y}_2$ are independent sample means. Then

$$\mathbb{E}[\hat{v}] = \mathbb{P}(\bar{Y}_1 \leq a) \cdot a + (1 - \mathbb{P}(\bar{Y}_1 \leq a)) \cdot \mathbb{E}[\bar{Y}_2] \leq \max(a, \mathbb{E}[Y]),$$

with strict inequality in general. This logic applies recursively at every node in the random tree, resulting in a global downward bias in $\hat{v}_0$.

Intuitively, this estimator corresponds to a feasible policy that refrains from using

future information in real-time exercise decisions. As such, it always produces a value that is less than or equal to the optimal stopping value, which relies on perfect foresight for valuation. $\mathbb{E}[\hat{v}_0] \leq V_0(X_0)$.

2.3.4 Asymptotic Convergence

Under mild moment conditions (in particular $\mathbb{E}[h_i(X_i)] < \infty$ for all i), both the high and low estimators converge to the true option value in probability as the branching factor $b \rightarrow \infty$. More precisely,

$$\lim_{b \rightarrow \infty} \hat{v}_0 = V_0(X_0) = \lim_{b \rightarrow \infty} \hat{V}_0$$

This is *convergence in probability* (and in fact also in L^1), meaning for every $\varepsilon > 0$

$$\mathbb{P}(|\hat{V}_0 - V_0(X_0)| > \varepsilon) \rightarrow 0, \quad \mathbb{P}(|\hat{v}_0 - V_0(X_0)| > \varepsilon) \rightarrow 0.$$

Why does it hold? At each node, the continuation value is estimated by the sample mean of b i.i.d. draws, so by the Law of Large Numbers it converges to the true conditional expectation. The choice to exercise or continue is then a function of this estimated continuation value. Using the contraction property

$$|\max(a, c_1) - \max(a, c_2)| \leq |c_1 - c_2|,$$

one shows by backward induction that the supremum norm of the error at time i is bounded by the error at time $i + 1$. As $b \rightarrow \infty$ every node's continuation-value estimate converges, hence the dynamic-programming recursion converges node-wise, yielding convergence in probability (and in L^1) at the root.

2.4 Depth-First Processing (DFS)

As the number of time steps m and the branching factor b increase, the total number of nodes in the tree grows exponentially (specifically, $\sum_{i=0}^m b^i$). This makes it impossible to store the entire tree in memory for large values of m or b . To deal with this, we use a method called *depth-first processing* (DFS).

Instead of building and storing all the nodes at once, DFS explores one branch at a time all the way to the end (a terminal node), keeping only the current path in memory. This method ensures correctness of high/low estimators while enabling

practical pricing for deeper trees.

A naive implementation of the random tree method generates all m^b nodes (over m steps with branching parameter b) and computes high and low estimators recursively. By noting that the high and low values at each node depend only on the subtree rooted at that node, we can dramatically reduce the storage requirements. It is never necessary to store more than $mb + 1$ nodes at a time.

The main idea: each node can be labeled by a sequence like $j_1 j_2 \dots j_i$, where each j_k tells us which branch we took at step k . With DFS, we don't build all branches from a node at once. Instead, we follow one branch all the way down, process it, and then move to the next one.

Take a 4-step tree as an example. We might start with: 1, 11, 111, 1111, which is one full path. Once we reach 1111, we generate : 1112, $\dots$, 111 b , to compute the estimators at node 111, and then discard those child nodes. Then we move on to 112 and repeat for: 1121, 1122, $\dots$, 112 b , and so on, until we finish all children of 11. Then we go back up to 1, repeat for nodes 12 to 1 b , then process nodes 2 through b , and finally reach the root.

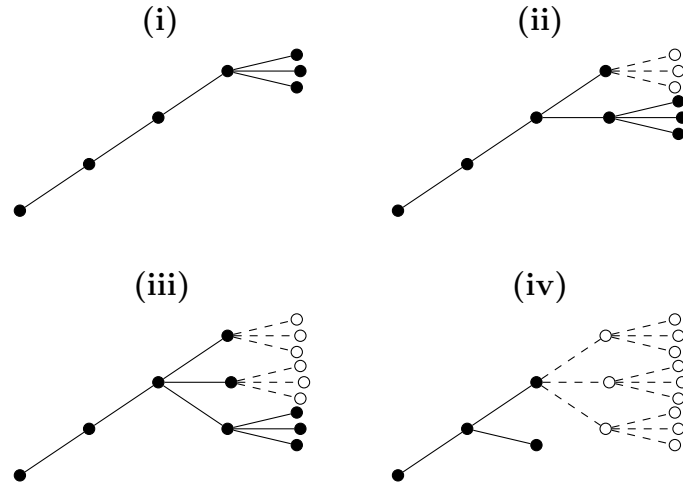


Figure 2: Four stages of Depth-first processing of tree are illustrated for $m = 4$ and $b = 3$. Dashed lines indicate branches previously generated, processed, and discarded. Solid node indicate nodes currently in memory. Open node indicate nodes previously processed and no longer stored.

3 Implementation

In this section we describe how the depth-first Monte Carlo tree algorithm for American option pricing is implemented. Here we focus on code structure, data flow, and on-the-fly tree-generation with minimal storage.

Main driver (`price_american_dfs`)

1. Input parameters: `S0` (initial price), `K` (strike), `r` (risk-free rate), `sigma` (volatility), `T` (maturity), `m` (exercise dates), `b` (branching factor), `n` (repetitions), `type` ("put" or "call").
2. Pre-computations: set $dt = \frac{T}{m}$, $discount = e^{-r dt}$, and define `payoff(S)` to return the intrinsic value.
3. Monte Carlo loop: for $i = 1, \dots, n$, call `simulate_node(1, S0)`, then store its returned `high` and `low` in `high_vals[i]` and `low_vals[i]`.
4. Post-processing: compute sample means $\bar{V}_{\text{high}}, \bar{V}_{\text{low}}$, standard errors $s_{\text{high}}/\sqrt{n}$, $s_{\text{low}}/\sqrt{n}$, and form the 95% confidence interval

$$\left[\bar{V}_{\text{low}} - z_{.975} \frac{s_{\text{low}}}{\sqrt{n}}, \bar{V}_{\text{high}} + z_{.975} \frac{s_{\text{high}}}{\sqrt{n}} \right].$$

Depth-First Recursion(`simulate_node(step, S)`)

1. Terminal check: if `step > m`, return (h, h) where $h = \text{payoff}(S)$.
2. Branch generation: for $k = 1 \dots b$: draw $Z^{(k)} \sim N(0, 1)$, set

$$S^{(k)} = S \exp\left((r - \frac{1}{2}\sigma^2) dt + \sigma\sqrt{dt} Z^{(k)}\right),$$

recurse to obtain $(V_{\text{high}}^{(k)}, V_{\text{low}}^{(k)})$.

3. Immediate-exercise value: $h = \text{payoff}(S)$.
4. High estimator: compute $\hat{C}_{\text{high}} = discount \frac{1}{b} \sum_k V_{\text{high}}^{(k)}$, then $\hat{V}_{\text{high}} = \max(h, \hat{C}_{\text{high}})$.
5. Low estimator: apply the leave-one-out test on the $\{V_{\text{low}}^{(k)}\}$ to decide exercise vs. continuation, then average to get $\hat{V}_{\text{low}}$.
6. Return: $(\hat{V}_{\text{high}}, \hat{V}_{\text{low}})$.

This depth-first approach ensures only $O(mb)$ nodes reside in memory at any time, while the outer loop builds independent replicates whose sample statistics yield the final option price and its confidence bounds.

4 Simulation results

In this section we first illustrate a single random-tree valuation of an American call showing high and low estimators at each node. We then turn to two sensitivity studies: one on how the branching factor b influences our stochastic bounds, and one on how the number of independent trees n (i.e. Monte Carlo replications) affects estimator precision.

4.1 Single Random-Tree Valuation of an American Call Option

This graphic depicts high and low estimators at each node for an American call option (strike $K = 100$, initial asset price $S_0 = 100$) using a random-tree valuation. While demonstrated for a call, the model adapts readily to put options. For clarity, we simplify with discount factor $\delta = 1$ (generalizable to cases like $\delta = \exp(-r\frac{T}{m})$). The tree is generated and traversed to compute all node estimators.

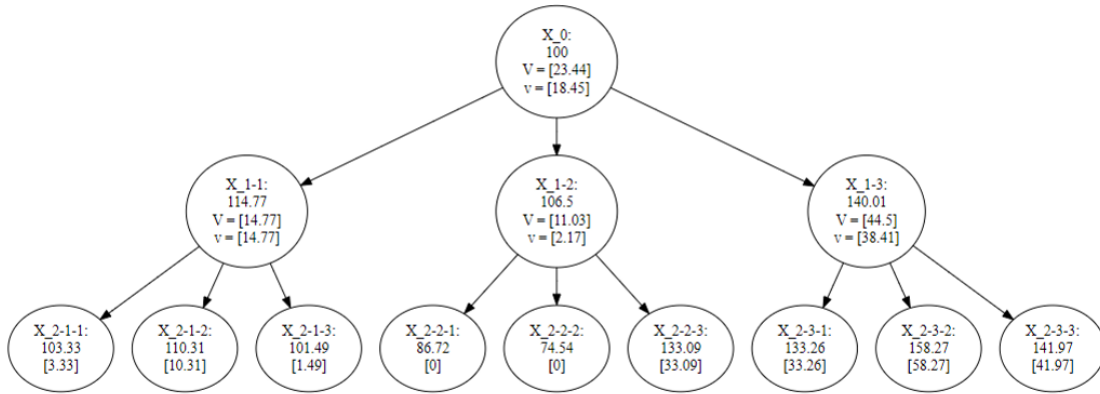


Figure 3: High and low estimators on a single random tree. Each node X_{i-j-k} shows the simulated underlying price (top), the high estimator V (middle), and the low estimator v (bottom).

These results were made using the programming code in Appendix 1. The next sections show studies that check how the branching ratio b and the number of trees

affect the accuracy of pricing American options with the random tree method. These studies use the results from the code in Appendix 2.

4.2 Effect of Branching Factor b

To study how the branching factor b influences estimator precision, we fix $S_0 = 100$, $K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$, $m = 3$, $n = 500$ and rerun the random-tree simulation for $b = 5, 10, 15, 20$. We collect the high/low bounds from each of the n replications.

4.2.1 Distribution of CI Widths

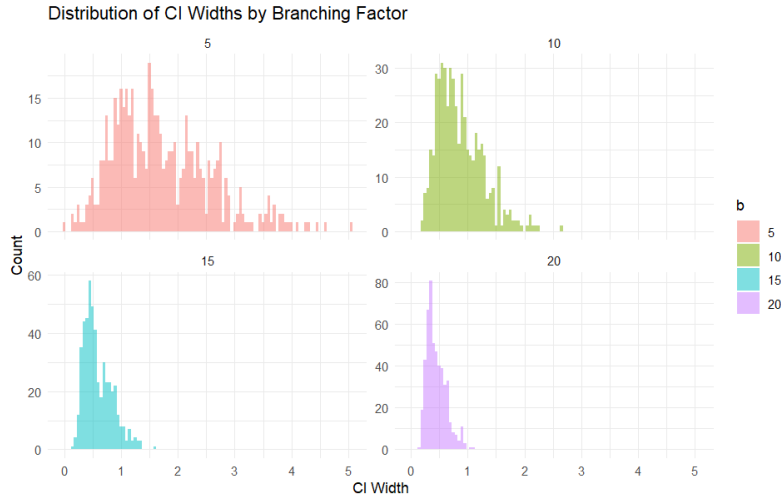


Figure 4: Empirical histograms of CI width = (high) – (low) for branching factors $b = 5, 10, 15, 20$.

Figure 4 shows that each histogram displays the distribution of resulting confidence-interval widths (high minus low bound) across the 500 trials.

As theory predicts, a larger branching factor drives both estimators toward the true price, shrinking the gap. Specifically, when b increases:

- The bulk of the CI-width distribution shifts left (toward zero).
- The variance of the width decreases, so large outliers become rare.

This agrees with the convergence result that as $b \rightarrow \infty$ the high and low estimators both become consistent and their difference vanishes.

4.2.2 High/Low Trajectories



Figure 5: High (solid) and low (dashed) price estimates vs. replication index for branching factors $b = 5, 10, 15, 20$.

Figure 5 plots, for each replication $j = 1, \dots, 500$, the high estimator (solid line) and low estimator (dashed line) of the American-call price under each value of b . In every trial, the high estimate lies above the low estimate, illustrating the inherent upward vs. downward bias. As b grows, the vertical gap between solid and dashed curves narrows, confirming that both estimators converge to the same limit. Overall, both sets of plots agree quantitatively with the theory of random-tree pricing: the high/low pair form valid stochastic bounds that bracket the true option value, and increasing b yields tighter intervals converging to the correct price.

4.3 Effect of Number of Trees n

To assess how the number of Monte Carlo trees n affects estimator precision, we now fix $S_0 = 100$, $K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$, $m = 3$, $b = 10$ and rerun for $n = 100, 500, 1000, 5000$, again collecting the high/low bounds from each replication.

4.3.1 CI Widths vs. n

Figure 6 (left) shows histograms of the CI widths. When $n = 100$, the histogram is wide and centered around larger values—indicating high variability in the Monte

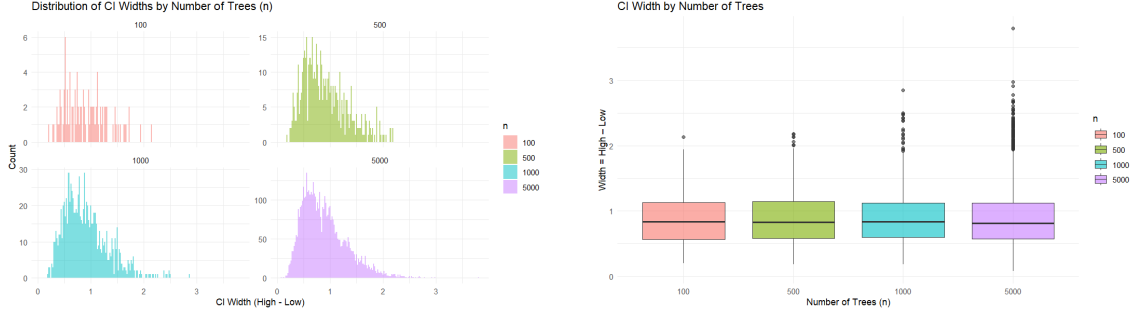


Figure 6: Left: Histograms of CI width = high – low for different sample sizes $n = 100, 500, 1000, 5000$. Right: Corresponding boxplots of CI width versus n .

Carlo estimators. As n increases, the histograms progressively shift left and become more concentrated. This narrowing distribution reflects reduced sampling error.

Figure 6 (right) presents boxplots of CI width against n , further emphasizing the shrinking interquartile range and median. The trend is clear: larger sample sizes yield narrower and more stable confidence intervals.

Theoretical Interpretation. This behavior is entirely consistent with the theory of random-tree estimators:

- The continuation-value estimators are averages over n random trees, so by the Central Limit Theorem the standard error shrinks as $1/\sqrt{n}$.
- Consequently, the expected width of the confidence interval behaves approximately as

$$\mathbb{E}[\text{width}] \approx z_{0.975} \frac{2\sigma}{\sqrt{n}},$$

where σ is the limiting standard deviation of the width across samples.

- This implies that doubling n should reduce the width by a factor of about $1/\sqrt{2} \approx 0.71$, and increasing n by 25 should reduce the width to about $1/5$.

These trends reinforce the conclusion that increasing n boosts estimator reliability and narrows the uncertainty in the price estimate—just as predicted by theory.

5 Conclusions

In this paper, we have considered in detail the Random Tree method for pricing American options proposed by Broadie and Glasserman. This approach combines Monte Carlo simulation with dynamic programming on trees, providing both upper and lower biased estimates of the option price. The key advantage of the method is the ability to construct a confidence interval around the true price without the need for analytical approximations or regression models. Such a built-in approximation ensures transparency of the results and increases confidence in the financial decisions made.

The implementation of the algorithm in the R language clearly demonstrates the stages of constructing and estimating a random tree. We showed how the estimates are calculated at each node of the tree and conducted numerical experiments investigating the effect of branching factors and the number of simulated trees on the accuracy of the results. As theory shows and simulations confirm, an increase in the number of branches leads to narrowing of the confidence intervals and closer estimates to the true price. An increase in the number of independent trees reduces variance due to the suppression of Monte Carlo noise. In addition, the use of depth-first processing allows for a significant reduction in memory requirements and makes the method applicable to resource-constrained problems.

The strength of the method lies in its simplicity, adaptability, and generality: it is applicable to a variety of stochastic processes without restrictions on the shape of the distribution or assumptions about the model structure. At the same time, theoretical guarantees of convergence confirm its robustness under moderate conditions.

Despite the exponential growth of computational costs with an increasing number of exercise dates, the method is particularly well suited for Bermuda-type options with a limited number of discrete exercise rights, such as callable bonds and energy contracts.

The random tree method thus represents a powerful and robust tool in the financial engineer's toolbox, effectively combining rigorous theory with the practical capabilities of computational modelling and providing a quantitatively justified estimate of uncertainty.

Bibliography

- [1] Paul Glasserman. “Monte Carlo Methods in Financial Engineering”. In: vol. 53. *Stochastic Modelling and Applied Probability*. Section 8.3: Pricing American Options. Springer New York, 2003-08-07. Chap. 8, pp. 421–479.
- [2] Mark Broadie and Paul Glasserman. “A stochastic mesh method for pricing high-dimensional American options”. In: *Journal of Computational Finance* 3.2 (1997), pp. 1–27.
- [3] Christoph Glur. *data.tree: General Purpose Hierarchical Data Structure*. R package version 0.9.9. 2024. URL: <https://cran.r-project.org/package=data.tree>.
- [4] Rich Iannone. *DiagrammeR: Graph/Network Visualization*. R package version 1.0.10. 2024. URL: <https://cran.r-project.org/package=DiagrammeR>.

Appendix 1

Below is the R code, that we used to generate and estimate American options using Random Tree method:

```
1 library(data.tree)
2
3 # Function to simulate next stock price under GBM
4 simulate_next <- function(S_current) {
5   S_current * exp((r - 0.5 * sigma^2) * dt + sigma * sqrt
6     (dt) * rnorm(1))
7 }
8
9 payoff_put <- function(K, S){
10   return(max(K - S, 0))
11 }
12
13 payoff_call <- function(K, S){
14   return(max(S - K, 0))
15 }
16
17 # function that returns a data.tree structure
18 generate_random_tree <- function(S, depth, label = "") {
19   node <- Node$new(paste0("X_", depth, label, ":\n",
20     round(S, 2)))
21   node$S <- S
22
23   if (depth < m) {
24     for (j in 1:b) {
25       S_next <- simulate_next(S)
26       child_label <- paste0(label, "-", j)
27       child_node <- generate_random_tree(S_next, depth +
28         1, child_label)
29       node$AddChildNode(child_node)
30     }
31   } else {
32     node$payoff <- payoff_call(K, S)
33   }
34   return(node)
35 }
```

```

32 }
33
34 calculate_estimators <- function(node) {
35   if (node$isLeaf) {
36     node$V_high <- payoff_call(K, node$S)
37     node$V_low <- payoff_call(K, node$S)
38     node$name <- paste0(node$name, "\n[", round(node$V_
39       high,2), "]"")
40   } else {
41     immediate <- payoff_call(K, node$S)
42
43     # Recursive calls to children
44     child_vals <- lapply(node$children, function(child) {
45       calculate_estimators(child)
46       list(V_high = child$V_high, V_low = child$V_low)
47     })
48
49     # Compute high estimator
50     child_highs <- sapply(child_vals, function(x) x$V_
51       high)
52     continuation_high <- mean(child_highs) * discount
53     node$V_high <- max(immediate, continuation_high)
54
55     # Compute low estimator
56     child_lows <- sapply(child_vals, function(x) x$V_low)
57     b <- length(child_lows)
58     val_vec <- numeric(b)
59
60     for (k in 1:b) {
61       continuation_k <- mean(child_lows[-k]) * discount
62       val_vec[k] <- ifelse(continuation_k <= immediate,
63         immediate, child_lows[k] *
64         discount)
65     }
66
67     node$V_low <- mean(val_vec)
68     node$name <- paste0(node$name, "\nV_L= ", round(node$
69       V_high,2), "]"",

```

```
66         "\nv_=" , round(node$V_low,2), "  
67         ]")  
68     }  
69     return(list(V_high = node$V_high, V_low = node$V_low))  
70 }  
71  
72  
73 # Parameters  
74 S0 <- 100           # Initial stock price  
75 K <- 100            # Strike price  
76 r <- 0.05           # Risk-free interest rate  
77 sigma <- 0.2        # Volatility  
78 T <- 1              # Time to maturity (in years)  
79 m <- 2              # Number of exercise opportunities (  
    excluding time 0)  
80 b <- 3              # Branching parameter  
81  
82 dt <- T / m         # Time step  
83 discount <- 1 # Discount factor per step  
84  
85  
86 tree <- generate_random_tree(S0, 0)  
87 print(tree, "S")  
88  
89 calculate_estimators(tree)  
90  
91 # Plot the tree  
92 library(DiagrammeR)  
93 ToDiagrammeRGraph(tree) %>% DiagrammeR::render_graph()
```

Appendix 2

```

1 library(ggplot2)
2
3 # Function returning high/low estimators for American
  option via random trees
4 price_american_dfs <- function(S0, K, r, sigma, T, m, b,
  n, type = "put") {
5   dt      <- T/m; discount <- exp(-r*dt)
6   payoff  <- function(S) if (type=="put") pmax(K-S,0)
    else pmax(S-K,0)
7
8   simulate_node <- function(step, S) {
9     if (step > m) {
10       v <- payoff(S)
11       return(list(high = v, low = v))
12     }
13     child_high <- child_low <- numeric(b)
14     for (k in seq_len(b)) {
15       Z      <- rnorm(1)
16       S_next <- S * exp((r-0.5*sigma^2)*dt + sigma*sqrt(
17         dt)*Z)
18       res    <- simulate_node(step+1, S_next)
19       child_high[k] <- res$high; child_low[k] <- res$low
20     }
21     exc_val  <- payoff(S)
22     cont_high <- discount * mean(child_high)
23     high_val  <- max(exc_val, cont_high)      # high
24     estimator
25
26     g <- numeric(b)
27     for (k in seq_len(b)) {
28       cont_k    <- discount * mean(child_low[-k])
29       g[k]      <- if (cont_k <= exc_val) exc_val else
        discount*child_low[k]
30     }
31     low_val <- mean(g)                        # low
32     estimator

```

```

31     list(high = high_val, low = low_val)
32   }
33
34   high_vals <- low_vals <- numeric(n)
35   for (i in seq_len(n)) {
36     root      <- simulate_node(1, S0)
37     high_vals[i] <- root$high
38     low_vals[i]  <- root$low
39   }
40
41   m_high <- mean(high_vals); s_high <- sd(high_vals); se_
     h <- s_high/sqrt(n)
42   m_low  <- mean(low_vals);  s_low  <- sd(low_vals); se_
     l <- s_low/sqrt(n)
43   z      <- qnorm(0.975)
44
45   list(
46     high_est      = m_high,
47     high_moe      = z * se_h,
48     low_est       = m_low,
49     low_moe       = z * se_l,
50     combined_CI   = c(lower = m_low - z*se_l, upper = m_
         high + z*se_h),
51     high_vals     = high_vals,
52     low_vals      = low_vals
53   )
54 }
55
56 # Sweep over branching factor
57 bs <- c(5, 10, 15, 20)
58 results_b <- data.frame()
59 for (bval in bs) {
60   out <- price_american_dfs(100, 100, 0.05, 0.2, 1, 3,
        bval, 100, "call")
61   df <- data.frame(sim = seq_along(out$high_vals),
62                     high = out$high_vals,
63                     low  = out$low_vals,
64                     b     = factor(bval))

```

```

65   df$width <- df$high - df$low
66   results_b <- rbind(results_b, df)
67 }
68 p1_b <- ggplot(results_b, aes(width, fill = b)) +
69   geom_histogram(position="identity", alpha=0.5, binwidth
70     =0.05) +
71   facet_wrap(~b, scales="free_y") +
72   labs(title="CI Widths by Branching Factor", x="CI Width
73     ", y="Count") +
74   theme_minimal()
75 print(p1_b)
76
77 p2_b <- ggplot(results_b, aes(sim)) +
78   geom_ribbon(aes(ymin=low, ymax=high, fill=b), alpha
79     =0.2) +
80   geom_line(aes(y=high, color=b), alpha=0.7) +
81   geom_line(aes(y=low, color=b), linetype="dashed", alpha
82     =0.7) +
83   facet_wrap(~b) +
84   labs(title="Estimators by Branching Factor", x="
85     Simulation", y="Option Value") +
86   theme_minimal()
87 print(p2_b)
88
89 # Sweep over number of trees
90 ns <- c(100, 500, 1000, 5000)
91 bval <- 10
92 results_n <- data.frame()
93 for (nval in ns) {
94   out <- price_american_dfs(100, 100, 0.05, 0.2, 1, 3,
95     bval, nval, "call")
96   df <- data.frame(sim = seq_along(out$high_vals),
97     high = out$high_vals,
98     low = out$low_vals,
99     n = factor(nval))
100   df$width <- df$high - df$low
101   results_n <- rbind(results_n, df)
102 }

```



```
97 p1_n <- ggplot(results_n, aes(width, fill = n)) +
98   geom_histogram(position="identity", alpha=0.5, binwidth
99     =0.02) +
100   facet_wrap(~n, scales="free_y") +
101   labs(title="CI Widths by Number of Trees", x="CI Width"
102     , y="Count") +
103   theme_minimal()
104 print(p1_n)
105
106 library(dplyr)
107 agg_df <- results_n %>%
108   group_by(n) %>%
109   summarise(widths = list(width)) %>%
110   unnest(widths)
111 ggplot(agg_df, aes(x=n, y=widths, fill=n)) +
112   geom_boxplot(alpha=0.6) +
113   labs(title="CI Width by Number of Trees", x="Number of
114     Trees", y="Width") +
115   theme_minimal()
```